

HOTEL MANAGEMENT SYSTEM DATABASE COURSE FINAL

Project Details

Item	Details
Project	Hotel Management System
Course	Database Course
Students	Hussein Joudeh / Mohammad Abdulrahman
Date	16/5/2026
Technologies	Java 17, Java Swing, JDBC, MySQL, Maven, FlatLaf

Table of Contents

1. Introduction
2. Problem Statement
3. Project Objectives
4. System Users and Roles
5. Database Design Overview
6. Main Relationships
7. Important Database Features
8. Normalization Summary
9. Application Implementation
10. Core Modules
11. Transactions
12. Stored Procedures and Triggers
13. Reports
14. Testing and QA
15. Conclusion
16. References

1. Introduction

Hotels manage many interrelated business processes: guest profiles, room inventory, reservations, in-stay services, billing, and staff access control. When these processes rely on paper records or disconnected spreadsheets, data becomes inconsistent, reservations can overlap, and financial totals are difficult to verify. A database-backed management system centralizes this information in one relational schema so that every operation - booking a room, checking in a guest, adding a service, or recording a payment - reads and writes consistent data.

The Hotel Management System developed in this project is a desktop application that connects a Java Swing user interface to a MySQL database through JDBC. The system demonstrates relational modeling, integrity constraints, views, stored procedures, triggers, transactional workflows, and practical data access from application code. The implementation is intentionally focused on core hotel operations rather than on web deployment or enterprise-scale features.

The database script set is maintained under the database/ folder (schema.sql, procedures.sql, triggers.sql, views.sql, and seed.sql). The runnable Java source is maintained under src/main/java and is built with Maven using pom.xml.

2. Problem Statement

- Manual reservations and double booking: Reception staff may record bookings in notebooks or separate files. Without a single source of truth, two guests can be assigned the same room for overlapping dates.
- Room and inventory tracking: Rooms belong to types with different prices and capacities. Each room has a current status such as Available, Occupied, Cleaning, Maintenance, or Inspection.
- Guest records and identity documents: Guest contact details and identification documents must be stored securely and linked to reservations. Duplicate emails or document numbers should be rejected.
- Packages and optional services: Hotels often sell packages that bundle services, and guests may request individual services during a stay. Historical prices must be preserved.
- Check-in, check-out, and invoicing: At arrival, a confirmed reservation becomes checked in and the room becomes occupied. At departure, the system generates an invoice and sets the room to Cleaning.
- Payment tracking: Invoices may be paid in full or in parts. The system must track payments per invoice and update invoice status without manual recalculation errors.
- Operational reporting: Management needs summaries such as daily check-ins, revenue by period, occupancy, and popular services or packages.
- User access and roles: The application distinguishes Admin, Manager, and Reception roles, with Admin-only user management.

3. Project Objectives

- Manage guest profiles and identity documents.
- Manage room types, rooms, floors, prices, capacity, and room status.
- Create reservations with confirmation numbers, dates, optional packages, and assigned rooms.
- Prevent overlapping room reservations for active statuses.
- Support check-in and check-out workflows with room status updates.

- Generate invoices from room, package, and service price snapshots.
- Record partial or full payments and maintain invoice balances.
- Provide reports for operations and revenue.
- Manage users and roles with bcrypt-hashed passwords.
- Demonstrate JDBC features such as PreparedStatement, ResultSet, ResultSetMetaData, CallableStatement, transactions, stored procedures, triggers, and views.

4. System Users and Roles

The Users table stores staff accounts with a role of Admin, Manager, or Reception. Passwords are stored as bcrypt hashes; plain-text passwords are never saved in the database.

Role	Responsibilities	Modules Accessible
Admin	Full system oversight, user management, password reset, and account activation/deactivation.	All modules, including the Admin-only Users module.
Manager	Operational supervision, reports, room/service/package configuration, and monitoring hotel activity.	Dashboard and all operational modules except Users.
Reception	Front-desk operations such as guest management, reservations, check-in/check-out, payments, and service requests.	Operational modules only; no Users module.

Demo Accounts for Testing

Role	Email	Password
Admin	admin@hotel.com	Admin123!
Manager	manager@hotel.com	Manager123!
Reception	reception@hotel.com	Reception123!

Role enforcement is implemented in the application layer. For example, only users with the Admin role can access the Users module. The database stores each user role, and the application uses this role after login to control authorization and module visibility.

5. Database Design Overview

The logical database name used during development is `hotel_management_system`. For course server deployment, a consolidated import script targets the existing database `hotel_management_system_2`.

#	Table	Purpose
1	Users	Staff accounts, roles, bcrypt password hashes
2	Guest	Guest personal and contact information
3	RoomType	Room categories, base price, capacity, amenities
4	Room	Physical rooms linked to a room type and status
5	Reservation	Booking header: guest, dates, status, confirmation number

#	Table	Purpose
6	ReservationItem	Room and optional package assigned to a reservation with price snapshots
7	Package	Optional stay packages
8	Service	Hotel services such as restaurant, spa, transport, laundry, and other services
9	PackageService	Many-to-many link between Package and Service
10	ServiceRequest	Services requested for a reservation during a stay
11	Invoice	Bill header per reservation
12	InvoiceItem	Line items on an invoice
13	Payment	Payments applied to an invoice
14	GuestDocument	Identity documents linked to a guest
15	RoomStatusLog	Audit trail of room status changes

The full DDL, including table creation order, foreign keys, constraints, and indexes, is defined in `database/schema.sql`. Sample operational data is stored in `database/seed.sql`.

6. Main Relationships

- Guest1:MReservation - eachreservationbelongs to one guest, while a guest may have many reservations.
- Reservation 1:M ReservationItem - reservation items store room and optional package assignment details.
- RoomType 1:M Room - each room references one room type.
- Room 1:M RoomStatusLog - every status change can be recorded with old status, new status, user, note, and timestamp.
- Package M:N Service through PackageService - bundled packages can include multiple services and services can belong to multiple packages.
- Reservation 1:M ServiceRequest - guests may request services during a stay.
- Reservation 1:1 Invoice - each reservation has at most one invoice, created at check-out.
- Invoice 1:M InvoiceItem - invoice line items describe room nights, packages, and services.
- Invoice 1:M Payment - multiple payments may apply to one invoice.
- Guest 1:M GuestDocument - document records belong to guests.
- Users 1:M Reservation and Users 1:M RoomStatusLog - staff users create bookings and status log entries.

7. Important Database Features

- Primarykeys:Everytableusesaclearprimarykey.PackageServiceuses a composite primary key on `package_id` and `service_id`.
- Foreign keys: Foreign keys enforce valid references and prevent orphaned operational data.
- Constraints: Unique constraints, domain checks, positive prices and quantities, valid statuses, and date logic support data quality.

- Indexes: Indexes support availability checks, reporting, reservations, payments, and foreign key joins.
- Views: Five views simplify dashboards and reporting: v_reservation_summary, v_room_status_board, v_invoice_balance, v_daily_operations, and v_service_revenue.
- Stored procedures: Three procedures are defined: sp_get_available_rooms, sp_get_invoice_balance, and sp_record_room_status_change.
- Trigger: trg_payment_update_invoice_status updates Invoice.status after payment insertion.
- Seed data: The seed file loads demo users, room types, rooms, guests, services, packages, reservations, invoices, payments, and room status logs.

8. Normalization Summary

The database design follows First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). A separate normalization document provides the detailed analysis.

- 1NF: all attributes are atomic and repeating groups are separated into related tables.
- 2NF: non-key attributes depend on the whole key. This is especially important for junction tables such as PackageService.
- 3NF: non-key attributes depend only on the key of their own table. Room type details are stored in RoomType rather than duplicated in Room.
- Intentional snapshots: room, package, and service price snapshots preserve billing history and prevent old invoices from changing when master prices change.

9. Application Implementation

The application is a Java 17 desktop program built with Maven. The entry point is Main.java, which applies the FlatLaf look-and-feel and opens the login window. The application uses explicit JDBC SQL access rather than an ORM.

Layer	Package / Files	Responsibility
UI	com.hotelmanagement.ui	Swing frames, panels, dialogs, navigation, and user interaction.
Service	com.hotelmanagement.service	Business rules, validation, transactions, and friendly errors.
DAO	com.hotelmanagement.dao	JDBC SQL access, PreparedStatement usage, and ResultSet mapping.
Model	com.hotelmanagement.model	Data transfer objects used by UI and services.
Config/Util	DatabaseConfig, DatabaseConnection	Database URL, credentials, DriverManager connection helper.

JDBC Usage

- Connection: DriverManager.getConnection is used through DatabaseConnection.
- PreparedStatement: parameterized queries are used for CRUD, filters, login, reports, and updates.
- ResultSet: DAO classes manually map rows into model objects.
- ResultSetMetaData: reports use dynamic column labels to build generic report tables.
- CallableStatement: the app calls sp_record_room_status_change when staff change room status.

10. Core Modules

Module	Summary
Login	Authenticates users using bcrypt password verification and blocks inactive users.
Dashboard	Displays statistics, recent reservations, and room status overview.
Guests and Guest Documents	Guest CRUD, search, and linked document management.
Rooms, Room Types, and Status Log	Room type and room management, status changes, and audit log.
Reservations	Create bookings, load available rooms, edit dates, confirm, and cancel with reason.
Services	Manage the hotel service catalog.
Packages	Manage packages and included services through PackageService.
Service Requests	Manage in-stay service requests and service price snapshots.
Check-In	Updates confirmed reservations to Checked-In and marks rooms Occupied.
Check-Out	Generates invoices and invoice items, updates reservation, and marks rooms Cleaning.
Payments	Records payments and shows database-updated invoice statuses.
Reports	Runs daily operations, occupancy, revenue, payment, services, and packages reports.
Users	Admin-only user management with password reset and safety rules.

11. Transactions

Critical workflows use JDBC transactions with `setAutoCommit(false)`, `commit()`, and `rollback()` to protect data consistency.

Workflow	Transaction Details
Reservation creation	Validates guest, date range, selected room, and availability; inserts Reservation and ReservationItem in one transaction.
Check-in	Verifies status and date eligibility, updates Reservation to Checked-In, sets actual_check_in, updates Room to Occupied, and inserts RoomStatusLog.
Check-out and invoice generation	Creates Invoice and InvoiceItem rows from stored snapshots, updates Reservation to Checked-Out, sets actual_check_out, updates Room to Cleaning, and logs the status change.
Payment insertion	Reads current balance, validates amount, inserts Payment, and relies on the payment trigger to update invoice status.

12. Stored Procedures and Triggers

Name	Purpose
sp_get_available_rooms	Returns rooms that are Available and not reserved for overlapping dates by active reservations.

Name	Purpose
sp_record_room_status_change	Reads current room status, validates the new status, updates Room, and inserts RoomStatusLog. The application calls it using CallableStatement.
sp_get_invoice_balance	Returns invoice total, total paid, balance due, and invoice status.
trg_payment_update_invoice_status	AFTER INSERT ON Payment trigger that recalculates total paid and sets Invoice.status to Unpaid, Partially-Paid, or Paid.

13. Reports

TheReportsmodule runs analytical queries using joins, grouping, aggregation, date filters, and ResultSetMetaData for dynamic table headers.

- Daily Check-Ins
- Daily Check-Outs
- Occupancy / Room Status
- Revenue by date range
- Payment Method Breakdown
- Popular Services
- Popular Packages

14. Testing and QA

Testing was planned and documented to verify both the database and desktop application before submission.

- Manual test cases: docs/manual-test-cases.md defines 86 structured test cases across the system.
- QA execution checklist: docs/qa-execution-checklist.md provides a shorter demo-aligned checklist.
- Automated harness: FinalQaHarness checks database connection, required tables, views, procedures, trigger, demo users, bcrypt hashes, availability, invoice totals, payment trigger behavior, reports, and QA residue cleanup.
- Final harness result: 15 passed, 0 failed.
- Server database upload: database/server_upload_hotel_management_system_2.sql was used to upload the schema, procedures, triggers, views, and seed data to the course database hotel_management_system_2.

Known limitations: this is an academic desktop system, not a production multi-user web platform. The current application supports one room per reservation, document file paths are stored as text, invoices are generated at check-out, and refund/payment deletion workflows are outside project scope.

15. Conclusion

The Hotel ManagementSystem project successfully combines a normalized MySQL relational database with a working Java Swing desktop application. The schema defines fifteen interrelated tables with primary and foreign keys, constraints, and indexes. Views, stored procedures, and triggers extend the database with reporting convenience, encapsulated operations, and automatic invoice status maintenance.

The application implements the main hotel workflows - guest and room management, reservations with overlap prevention, packages and services, check-in and check-out with invoicing, payments, reports, and Admin-only user management - using JDBC best practices including Connection, PreparedStatement, ResultSet, ResultSetMetaData, CallableStatement, and explicit transaction control.

Testing documentation and the automated FinalQaHarness result (15/15 checks passed) confirm that the system is ready for demonstration and submission. The server-ready script allows the instructor to restore or verify the full database on MySQL.

16. References

Reference	Description
README.md	Project overview, setup, JDBC alignment, demo accounts.
database/schema.sql	Tables, constraints, keys, and indexes.
database/seed.sql	Demo seed data.
database/procedures.sql	Stored procedures.
database/triggers.sql	Payment status trigger.
database/views.sql	Database views.
database/dml_operations.sql	Sample DML reference.
database/server_upload_hotel_management_system_2.sql	Server import bundle.
database/final_verification_checks.sql	SQL verification queries.
docs/manual-test-cases.md	Full manual test case catalog.
docs/qa-execution-checklist.md	Demo and QA checklist.
docs/dbeaver-restore-guide.md	DBeaver connection and restore guide.
docs/database-export-checklist.md	Pre-export verification.
src/main/java/com/hotelmanagement/Main.java	Main application entry.
src/main/java/com/hotelmanagement/util/DatabaseConnection.java	JDBC connection helper.
src/main/java/com/hotelmanagement/qa/FinalQaHarness.java	Automated database QA harness.
Oracle Java 17 Documentation	JDBC API.
MySQL 8 Reference Manual	SQL, triggers, procedures, and views.
Maven	Build and dependency management.
FlatLaf	Swing look-and-feel.
at.favre.lib bcrypt	Password hashing library.